

Mise en place d'un projet .NET 8 + Entity Framework Core avec Rider

1. Création du projet

1. Ouvrir Rider

Lancer JetBrains Rider et cliquer sur **New Solution**.

2. Type de projet

- Choisir **.NET Core / Console Application**.
- Framework : **.NET 8**.
- Nommer le projet : **EfCoreLibrary**.
- Choisir l'emplacement, puis **Create**.

2. Installation des packages NuGet

1. Dans l'**Explorateur de solutions**, clic droit sur le projet → **Manage NuGet Packages**.
2. Onglet **Browse** : rechercher et installer (**8.0.20 uniquement**) :
 - `Microsoft.EntityFrameworkCore`
 - `Microsoft.EntityFrameworkCore.Design`
 - `Microsoft.EntityFrameworkCore.SqlServer`
3. Vérifier qu'ils apparaissent dans l'onglet **Installed**.

3. Création des entités

1. Clic droit sur le projet → **Add** → **New Folder** → nommer **Models**.
2. Dans ce dossier, ajouter quatre **classes C#** : `Author`, `Book`, `User`, `Loan`.
3. Dans chaque classe, définir les propriétés et les **Data Annotations** :

Exemple pour **Book** :

```
public class Book
{
    [Key] public int Id { get; set; }
    [Required, MaxLength(255)] public string? Title { get; set; }
    [Required] public int AuthorId { get; set; }
    public int? ReleaseYear { get; set; }
    [Required, MaxLength(20)] public string? Isbn { get; set; }
    public Author? Author { get; set; }
}
```

4. Propriétés de navigation

Dans les mêmes classes, ajouter les relations :

- **Author.cs**

```
public List<Book>? Books { get; set; }
```

- **Book.cs**

```
public Author? Author { get; set; }
```

- **Loan.cs**

```
public Book? Book { get; set; }
public User? User { get; set; }
```

- **User.cs**

```
public List<Loan>? Loans { get; set; }
```

5. Contexte de base de données

Clic droit sur le projet → Add → Class

Ajouter une classe **LibraryDbContext** héritant de **DbContext** :

```
public class LibraryDbContext : DbContext
{
    // Tables représentées par les entités
    [3 usages]
    public DbSet<Book> Books { get; set; } // Table des livres
    [2 usages]
    public DbSet<Author> Authors { get; set; } // Table des auteurs
    [3 usages]
    public DbSet<Loan> Loans { get; set; } // Table des prêts
    [2 usages]
    public DbSet<User> Users { get; set; } // Table des utilisateurs

    // Configuration de la connexion à la base de données
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        string connectionString =
            "Server=romaric-thibault.fr;" + // Serveur SQL
            "Database=mathys_EfCoreLibrary;" + // Nom de la base
            "User Id=mathys;" + // Utilisateur
            "Password=Onto9-Cage-Afflicted;" + // Mot de passe
            "TrustServerCertificate=true;" // Accepte certificat auto-signé

        optionsBuilder.UseSqlServer(connectionString);
    }

    // Personnalisation du modèle (non utilisée ici)
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
    }
}
```

6. Génération des migrations et création de la BDD

Il faut retenir 2 commandes pour créer la base de données de manière automatique, en partant de la base de code.

- La commande pour générer les migrations :

dotnet ef migrations add --project [namespace] --context [DbContext] InitialDatabase

dotnet ef migrations add → Crée une nouvelle migration (plan des changements à apporter à la base).

--project [namespace] → Précise le projet contenant le code source et le DbContext.

--context [DbContext] → Indique le DbContext à utiliser.

InitialDatabase → Nom de la migration (ici première version de la base).

Résultat : un fichier de migration est créé, décrivant les tables, colonnes et relations.

- La commande pour appliquer la migration à la base de données :

dotnet ef database update --project [namespace] --context [DbContext]

dotnet ef database update → Applique toutes les migrations non appliquées à la base.

--project [namespace] → Précise le projet contenant le DbContext.

--context [DbContext] → Indique le DbContext à utiliser.

Résultat : la base de données est créée automatiquement avec toutes les tables définies dans la

migration.

7. Création de la base

1. Ouvrir le menu Database
2. Faire New → Connect to Database
3. Bien regarder que “Add data source manually” soit coché.
4. Choisir le type de SGBD
5. Entrer les informations de connexion
6. Tester la connexion
7. Aller dans le menu déroulant et sélectionner les infos souhaitées.

8. Manipulation des données (Program.cs)

Dans **Program.cs**, tester :

- Se connecter au context pour avoir accès à la BDD

```
// Connexion à la BDD
using var context = new LibraryDbContext();
```

La variable context permet la communication avec la BDD lors des ajout, modification, suppression, sauvegarde, etc.

- Insertion :

```
// === Création ===

// 1 - Création d'un auteur
var author = new Author {Name = "Verne", FirstName = "Jules"};
context.Authors.Add(author); // Ajout dans la BDD
context.SaveChanges(); //Save

// 2 - Création d'un livre pour l'auteur
var book = new Book
{
    Title = "L'Ile mystérieuse",
    AuthorId = author.Id, // Liaison à la ligne Jules Verne
    ReleaseYear = 1875,
    Isbn = "978-2080245434"
};
context.Books.Add(book);
context.SaveChanges();
```

Modification :

```
// === Modification ===  
  
// 1 - Modification de la date du livre crée à 2015  
// Récupérer le livre par son ID et changer l'année  
var bookToUpdate = context.Books.Find(book.Id);  
if (bookToUpdate != null)  
{  
    bookToUpdate.ReleaseYear = 2015;  
    context.SaveChanges();  
}
```

Lecture avec jointures :

```
// === Affichage ===  
  
// 1 - Affichage des emprunts  
// === Affichage des emprunts avec jointures (Include) ===  
var loans :IIncludableQueryable<Loan,User?> = context.Loans //DbSet<Loan>  
    .Include( navigationPropertyPath: l :Loan => l.Book).ThenInclude(b :Book? => b.Author)  
    .Include( navigationPropertyPath: l :Loan => l.User);  
  
foreach (var l :Loan in loans)  
{  
    Console.WriteLine(  
        $"Livres : {l.Book.Title} ({l.Book.ReleaseYear}) - " +  
        $"Auteur : {l.Book.Author.FirstName} {l.Book.Author.Name} - " +  
        $"Emprunteur : {l.User.FirstName} {l.User.Name} - " +  
        $"Date emprunt : {l.Date} - Retour prévu : {l.PlannedReturningDate}");  
}
```

Suppression : supprimer l'emprunt, l'utilisateur, le livre puis l'auteur.

```
// === Suppression ===  
context.Loans.Remove(loan);  
context.Users.Remove(user);  
context.Books.Remove(book);  
context.Authors.Remove(author);  
context.SaveChanges();
```

9. Vérifications

- Ouvrir l'onglet Database pour voir les données modifiées en temps réel.
- Utiliser l'onglet Run ou Debug pour exécuter et contrôler les sorties.